Experiment11

1) Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
#include <time.h>

#define NUM_THREADS 4
#define INCREMENTS  1000000

volatile long counter = 0;              /* shared variable, NO protection */

void *increment(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++) {
        long temp = counter;            /* 1. read shared value            */
        if (i % 250000 == 0)
            sched_yield();              /* let the scheduler switch thread */
        counter = temp + 1;             /* 3. write back (may be stale)    */
    }
    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];
    struct timespec t1, t2;

    clock_gettime(CLOCK_MONOTONIC, &t1);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &t2);

    long expected = (long)NUM_THREADS * INCREMENTS;
    double elapsed = (t2.tv_sec - t1.tv_sec) + (t2.tv_nsec - t1.tv_nsec) / 1e9;

    printf("Without synchronization (race condition)\n");
    printf("Threads         : %d\n", NUM_THREADS);
    printf("Expected value  : %ld\n", expected);
    printf("Actual value    : %ld\n", counter);
    printf("Lost updates    : %ld\n", expected - counter);
    printf("Time taken      : %.4f seconds\n", elapsed);
    if (counter != expected)
        printf("Result          : RACE CONDITION occurred!\n");
    else
        printf("Result          : correct this time (race not triggered)\n");
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp11.1.c
charith@DESKTOP-RBQQQSS:~$ gcc exp11.1.c -o exp11.1 -lpthread
charith@DESKTOP-RBQQQSS:~$ ./exp11.1
Without synchronization (race condition)
Threads         : 4
Expected value  : 4000000
Actual value    : 1000000
Lost updates    : 3000000
Time taken      : 0.0166 seconds
Result          : RACE CONDITION occurred!
charith@DESKTOP-RBQQQSS:~$
```

2) Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <sched.h>
#include <time.h>

#define NUM_THREADS 4
#define INCREMENTS  1000000

volatile long counter = 0;               /* shared variable               */
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;   /* protects counter */

void *increment(void *arg)
{
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&lock);       /* enter critical section         */
        long temp = counter;             /* 1. read shared value           */
        if (i % 250000 == 0)
            sched_yield();               /* scheduler may switch thread,   */
        counter = temp + 1;              /*    but others stay blocked     */
        pthread_mutex_unlock(&lock);     /* leave critical section         */
    }
    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];
    struct timespec t1, t2;

    clock_gettime(CLOCK_MONOTONIC, &t1);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);
    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &t2);

    long expected = (long)NUM_THREADS * INCREMENTS;
    double elapsed = (t2.tv_sec - t1.tv_sec) + (t2.tv_nsec - t1.tv_nsec) / 1e9;

    printf("With mutex synchronization\n");
    printf("Threads         : %d\n", NUM_THREADS);
    printf("Expected value  : %ld\n", expected);
    printf("Actual value    : %ld\n", counter);
```

```
    printf("Lost updates    : %ld\n", expected - counter);
    printf("Time taken      : %.4f seconds\n", elapsed);
    if (counter == expected)
        printf("Result          : CORRECT - no updates were lost\n");
    else
        printf("Result          : WRONG - race condition still present\n");
    pthread_mutex_destroy(&lock);
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp11.2.c
charith@DESKTOP-RBQQQSS:~$ gcc exp11.2.c -o exp11.2 -lpthread
charith@DESKTOP-RBQQQSS:~$ ./exp11.2
With mutex synchronization
Threads         : 4
Expected value  : 4000000
Actual value    : 4000000
Lost updates    : 0
Time taken      : 0.0851 seconds
Result          : CORRECT - no updates were lost
charith@DESKTOP-RBQQQSS:~$
```